

Statistics Statistical Modelling And
Data Analytics LAB
(DA-304P)

Faculty Name : Mr. Akshay Mool

Student Name : Yash Pandey

Roll No : 12214802722 (AIML II-B)

Semester : 6th Semester



Maharaja Agrasen Institute of Technology, PSP Area,

Sector – 22, Rohini, New Delhi - 110086



MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

Computer Science & Engineering Department

VISION

"To be centre of excellence in education, research and technology transfer in the field of computer engineering and promote entrepreneurship and ethical values."

MISSION

To foster an open, multidisciplinary and highly collaborative research environment for producing world-class engineers capable of providing innovative solutions to real-life problems and fulfil societal needs.

Department of Computer Science & Engineering

VISION

"To attain global excellence through education, innovation, research, and work ethics in the field of Computer Science and engineering with the commitment to serve humanity."

MISSION

M1 To lead in the advancement of computer science and engineering through internationally recognized research and education.

M2 To prepare students for full and ethical participation in a diverse society and encourage lifelong learning.

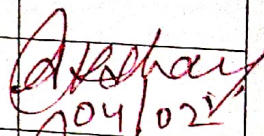
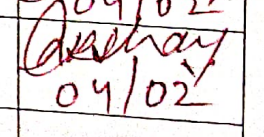
M3 To foster development of problem solving and communication skills as an integral component of the profession.

M4 To impart knowledge, skills and cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship in the field of computer science and engineering.

M5 To encourage faculty, student's networking with alumni, industry, institutions, and other stakeholders for collective engagement



LAB INDEX

S.No	Experiment	R1	R2	R3	R4	R5	Total Marks (10)	Remarks	Faculty Signature
		Is able to identify and define the objective of the given problem?	Is proposed design /procedure /algorithm solves the problem?	Has the understanding of the tool/programming language to implement the proposed solution?	Are the result(s) verified using sufficient test data to support the conclusions?	Individuality of submission?			
		2 Marks	2 Marks	2 Marks	2 Marks	2 Marks			
1	Implement basic Matrix operations								 04/02/21  04/02/21
2	Eigen Values and Vectors								
3	Gauss Eq ⁿ Solving								
4	Matrix operations								
5	Reduced Row Echelon form								
6	derivative Implementation								
7	Frequency Table In SPSS								
8	outlier dataset In SPSS								
9	Mutually Exclusive In SPSS								
10	Diagrams Using R - Module								
11	Correlation Using R								
12	Time Series Using R								

LAB INDEX

[illegible]

Lab Assessment Sheet

[illegible]

Lab Assessment Sheet

[illegible]

Lab Exercise-1. (21/01/2025)

Q :- Exercises to implement the basic matrix operations in Scilab?

Here are brief descriptions for each operation:

1. **Addition:** Matrix addition involves adding corresponding elements of two matrices of the same size. The result is a matrix of the same dimensions, where each element is the sum of the respective elements from the original matrices.

$$\begin{bmatrix} a_1 & b_1 \\ c_1 & d_1 \end{bmatrix} + \begin{bmatrix} a_2 & b_2 \\ c_2 & d_2 \end{bmatrix} = \begin{bmatrix} a_1 + a_2 & b_1 + b_2 \\ c_1 + c_2 & d_1 + d_2 \end{bmatrix}$$

2. **Subtraction:** Matrix subtraction is the operation of subtracting corresponding elements of two matrices of the same size. The result is a matrix of the same dimensions, where each element is the difference of the respective elements from the original matrices.

$$\begin{bmatrix} a_1 & b_1 \\ c_1 & d_1 \end{bmatrix} - \begin{bmatrix} a_2 & b_2 \\ c_2 & d_2 \end{bmatrix} = \begin{bmatrix} a_1 - a_2 & b_1 - b_2 \\ c_1 - c_2 & d_1 - d_2 \end{bmatrix}$$

3. **Multiplication:** Matrix multiplication involves multiplying the rows of the first matrix by the columns of the second matrix. The result is a new matrix, where each element is the dot product of the respective row and column.

$$\begin{bmatrix} a_1 & b_1 \\ c_1 & d_1 \end{bmatrix} \times \begin{bmatrix} a_2 & b_2 \\ c_2 & d_2 \end{bmatrix} = \begin{bmatrix} a_1a_2 + b_1c_2 & a_1b_2 + b_1d_2 \\ c_1a_2 + d_1c_2 & c_1b_2 + d_1d_2 \end{bmatrix}$$

4. **Determinant:** The determinant of a square matrix is a scalar value that can be computed from its elements. It provides important information about the matrix, such as whether it is invertible (non-zero determinant) or singular (zero determinant).

Matrix:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Determinant:

$$\det(A) = |A| = \begin{vmatrix} a & b \\ c & d \end{vmatrix}$$

CODE : -

// Define two matrices A and B

```
A = [1 2 3; 4 5 6; 7 8 9];
```

```
B = [9 8 7; 6 5 4; 3 2 1];
```

// Matrix Addition

```
disp("Matrix Addition (A + B):");
```

```
C_add = A + B;
```

```
disp(C_add);
```

// Matrix Subtraction

```
disp("Matrix Subtraction (A - B):");
```

```
C_sub = A - B;
```

```
disp(C_sub);
```

// Matrix Multiplication

```
disp("Matrix Multiplication (A * B):");
```

```
C_mult = A * B;
```

```
disp(C_mult);
```

// Matrix Transpose

```
disp("Transpose of Matrix A:");
```

```
A_transpose = A';          disp(A_transpose);
```

```
disp("Transpose of Matrix B:");
```

```
B_transpose = B';          disp(B_transpose);
```

// Determinant of Matrix A

```
disp("Determinant of Matrix A:");
```

```
det_A = det(A);             disp(det_A);
```

```
disp("Determinant of Matrix B:");
```

```
det_B = det(B);             disp(det_B);
```

OUTPUT :-

```
--> exec('C:\Users\maitl\Yash122.sce', -1)
```

```
"Matrix Addition (A + B):"
```

10.	10.	10.
10.	10.	10.
10.	10.	10.

```
"Matrix Subtraction (A - B):"
```

-8.	-6.	-4.
-2.	0.	2.
3.	6.	8.

```
"Matrix Multiplication (A * B):"
```

30.	24.	18.
84.	69.	54.
138.	114.	90.

```
"Transpose of Matrix A:"
```

1.	4.	7.
2.	5.	8.
3.	6.	9.

```
"Transpose of Matrix B:"
```

9.	6.	3.
8.	5.	2.
7.	4.	1.

```
"Determinant of Matrix A:"
```

```
5.
```

```
"Determinant of Matrix B:"
```

```
12.
```

Lab Exercise-2. (28/01/2025)

Q :- Exercise To find the Eigen-values and Eigen-vectors in Scilab?

Here are brief descriptions for EigenValue And EigrnVector:

1. Eigenvalues are special scalars associated with a matrix that indicate how the matrix stretches or compresses vectors along certain directions. They help in understanding the stability and transformation properties of a system. In real-world applications, eigenvalues are used in image processing, quantum mechanics, and machine learning.
2. Eigenvectors are the corresponding directions that remain unchanged (except for scaling) when the matrix is applied to them. They provide insight into the fundamental structure of transformations. Eigenvectors are widely used in Principal Component Analysis (PCA) to reduce dimensionality in data science.

$$\begin{aligned} Ax - \lambda x &= 0 \\ (A - \lambda I)x &= 0 \end{aligned}$$

CODE : -

```
n = input("Enter the size of the square matrix (n x n): ");
```

```
A = zeros(n, n);
```

```
for i = 1:n
```

```
    for j = 1:n
```

```
        A(i, j) = input("Enter element A(" + string(i) + "," + string(j) + "): ");
```

```
    end
```

```
end
```

```
trA = trace(A);    detA = det(A);
```

```
discriminant = trA^2 - 4 * detA;
```

```
lambda1 = (trA + sqrt(discriminant)) / 2;
```

```
lambda2 = (trA - sqrt(discriminant)) / 2;
```

```
disp("Eigenvalues:");
```

```
disp(lambda1);    disp(lambda2);
```

```
I = eye(n);
```

```
A_lambda1 = A - lambda1 * I;
```

```
v1 = A_lambda1(:,1);
```

```
v1 = v1 / norm(v1);
```

```
disp("Eigenvector corresponding to lambda1:");
```

```
disp(v1);
```

```
A_lambda2 = A - lambda2 * I;
```

```
v2 = A_lambda2(:,1);
```

```
v2 = v2 / norm(v2);
```

```
disp("Eigenvector corresponding to lambda2:");
```

```
disp(v2);
```

OUTPUT :-

```
Enter the size of the square matrix (n x n): 3
```

```
Enter element A(1,1): 1
```

```
Enter element A(1,2): 2
```

```
Enter element A(1,3): 3
```

```
Enter element A(2,1): 4
```

```
Enter element A(2,2): 5
```

```
Enter element A(2,3): 6
```

```
Enter element A(3,1): 7
```

```
Enter element A(3,2): 8
```

```
Enter element A(3,3): 9
```

```
"Eigenvalues:"
```

```
15.
```

```
0.
```

```
"Eigenvector corresponding to lambda1:"
```

```
-0.7172417
```

```
-0.5635471
```

```
-0.4098524
```

```
"Eigenvector corresponding to lambda2:"
```

```
0.1230915
```

```
0.4923660
```

```
0.8616404
```

```
--> |
```

Lab Exercise-3. (04/02/2025)

Q :- Exercises to solve equations by Gauss elimination, Jordan, Siedel Method in Scilab?

1. Gauss Elimination Method

The **Gauss Elimination Method** is a systematic approach used to solve systems of linear equations by transforming the given system into an upper triangular form using row operations. This method consists of two main steps: **forward elimination**, where coefficients below the leading coefficient (pivot) in each column are eliminated, and **back-substitution**, where the unknowns are determined by solving the resulting upper triangular system. However, for large systems, numerical stability issues may arise, requiring techniques like **partial pivoting** to improve accuracy.

CODE : -

```
A = input('Enter the matrix A: ');
b = input('Enter the vector b: ');
Aug = [A b];
disp('Augmented Matrix:');
disp(Aug);
n = size(A, 1);
for k = 1:n-1
    for i = k+1:n
        if Aug(k, k) == 0
            break;
        end
        factor = Aug(i, k) / Aug(k, k);
        Aug(i, k:n+1) = Aug(i, k:n+1) - factor * Aug(k,
k:n+1);
    end
end
end
```

```

disp('Upper Triangular Matrix:');
disp(Aug);

x = zeros(n, 1);
x(n) = Aug(n, n+1) / Aug(n, n);

for i = n-1:-1:1
    x(i) = (Aug(i, n+1) - Aug(i, i+1:n) * x(i+1:n)) / Aug(i, i);
end

disp('Solution:');
disp(x);

```

OUTPUT :-

```

Enter the matrix A: [1 -2 1;2 3 1;3 -1 2]

Enter the vector b: [4;5;6]

"Augmented Matrix:"
 1.  -2.   1.   4.
 2.   3.   1.   5.
 3.  -1.   2.   6.
"Upper Triangular Matrix:"
 1.  -2.   1.           4.
 0.   7.  -1.          -3.
 0.   0. -0.2857143    -3.8571429
"Solution:"
-6.5000000
 1.5000000
13.5000000

```

2. Gauss Jordan Method

The **Gauss-Jordan Method** is an extension of the Gauss Elimination Method used to solve systems of linear equations and find matrix inverses. It transforms the augmented matrix into **reduced row echelon form (RREF)** using row operations, where the leading coefficient (pivot) in each row is 1, and all other elements in the pivot column are zero. Unlike Gauss elimination, which stops at an upper triangular matrix and requires back-substitution, Gauss-Jordan elimination fully reduces the matrix, directly yielding the solution. This method is particularly useful in computational applications for solving equations and finding the inverse of a matrix efficiently.

CODE : -

```
// Input
A = input('Enter the matrix A: ');
b = input('Enter the vector b: ');

Aug = [A, b]; // Augmented matrix

disp(Aug);
n = size(A, 1);

for k = 1:n
    // Pivoting: Make the diagonal element 1
    if Aug(k, k) == 0 then
        disp("Division by zero detected.");
        break;
    end
    Aug(k, :) = Aug(k, :) / Aug(k, k); // Scale the k-th row

    // Make the elements below and above the pivot 0
    for i = 1:n
        if i ~= k then
            factor = Aug(i, k);
            Aug(i, :) = Aug(i, :) - factor * Aug(k, :);
        end
    end
end
```

```

        end
    end
end

disp('Reduced Row Echelon Form (RREF) matrix:');
disp(Aug);

x = Aug(:, $); // Extract the solution from the last column
disp('Solution:');
disp(x);

```

OUTPUT :-

```

Warning: file 'C:\Users\maitl\Desktop\gosjodan.sci' already opened in Scilab.
Enter the matrix A: [2, 1, -1; -3, -1, 2; -2, 1, 2];

Enter the vector b: [8; -11; -3];

    2.    1.   -1.    8.
   -3.   -1.    2.  -11.
   -2.    1.    2.   -3.
"Reduced Row Echelon Form (RREF) matrix:"
    1.    0.    0.    2.
    0.    1.    0.    3.
    0.    0.    1.   -1.
"Solution:"
    2.
    3.
   -1.
exec: Wrong number of output argument(s): 0 expected.

```

3. Gauss Siedel Method

The **Gauss-Seidel Method** is an iterative technique used to solve systems of linear equations, particularly for large sparse matrices. It improves upon the **Jacobi Method** by using updated values of variables as soon as they are computed, leading to faster convergence. The method starts with an initial guess and updates each variable sequentially using previously computed values within the same iteration. This approach often accelerates convergence compared to direct methods like **Gauss Elimination**, but it requires the coefficient matrix to be **diagonally dominant** or symmetric positive definite for guaranteed convergence. The Gauss-Seidel method is widely used in numerical analysis, engineering, and scientific computing due to its efficiency in solving large systems.

CODE : -

```
// Input The Matrix A And B ...
disp("Enter the coefficient matrix A:");
A = input("A = ");

disp("Enter the constant matrix b:");
b = input("b = ");

n = size(A, 1);
x = zeros(n, 1); // Initial guess
tolerance = 1e-6;
max_iterations = 100;

for iter = 1:max_iterations
    x_old = x;

    for i = 1:n
        sum = b(i);

        for j = 1:n
            if j ~= i
                sum = sum - A(i, j) * x(j);
```

```

        end
    end
    x(i) = sum / A(i, i);
end
    // Check for convergence
    if norm(x - x_old, "inf") < tolerance
        disp("Convergence reached.");
        break;
    end
end
disp("Solution:");
for i = 1:n
    mprintf("x%d = %.6f\n", i, x(i)); // Display solution
    with 6 decimal places
end

```

OUTPUT :-

```

    "Enter the coefficient matrix A:"
A = [2 3 -1; 4 1 2; 3 2 3]

    "Enter the constant matrix b:"
b = [5; 6; 7]

    "Solution:"

    "x1 = 1.000000"

    "x2 = 1.000000"

    "x3 = 1.000000"

--> |

```

Lab Exercise-4. (11/02/2025)

Q :- Exercises to implement the associative, commutative and distributive property in a matrix in Scilab & 'R' ?

A **matrix** is a rectangular array of numbers, symbols, or expressions arranged in rows and columns, commonly used in mathematics, computer science, and engineering. It is represented as $A=[a_{ij}]$ or $A=[a_{ij}]$, where a_{ij} denotes the element at the i th row and j th column. Matrices are widely used for solving linear equations, transformations in graphics, data representation in machine learning, and various applications in physics and economics. Operations like addition, multiplication, and inversion allow matrices to manipulate and process data efficiently in multiple dimensions.

Matrices play a crucial role in various fields due to their ability to represent and manipulate data efficiently. They are used in computer graphics for 2D and 3D transformations, in artificial intelligence and machine learning for handling datasets, and in physics for solving complex equations. In electrical engineering, matrices help analyze circuits, while in economics, they model financial systems and optimize resource allocation.

1. Associative Property

- For matrix addition: $(A+B)+C=A+(B+C)$.
- For matrix multiplication: $(A \times B) \times C = A \times (B \times C)$.
- This means that grouping does not affect the result in addition or multiplication.

2. Commutative Property

- For matrix addition: $A+B=B+A$.
- **Not valid for multiplication:** $A \times B \neq B \times A$.
- Addition is commutative, but matrix mult. doesn't always follow this property.

3. Distributive Property

- Left distributive: $A \times (B+C) = A \times B + A \times C$.
- Right distributive: $(A+B) \times C = A \times C + B \times C$.
- This property allows factorization or expansion when multiplying a matrix with a sum of matrices.

1. SCILAB

Scilab is an open-source, high-level programming language and computing environment designed for numerical computation and data analysis. It provides a rich set of mathematical functions, making it widely used in engineering, scientific research, and machine learning. Scilab supports matrix operations, algorithm development, signal processing, and simulation of dynamic systems. Its built-in visualization tools allow users to create graphs and plots for data representation. With its user-friendly interface and powerful computational capabilities, Scilab serves as an excellent alternative to MATLAB .

CODE : -

```
disp("Yash Pandey :: 12214802722");

A = [1, 2; 3, 4];
B = [5, 6; 7, 8];
C = [9, 10; 11, 12];

disp("Commutative Property:");
disp("A * B:");      disp(A*B);
disp("B * A:");      disp(B*A);
if AB == BA then
    disp("Matrix multiplication is commutative.");
else
    disp("Matrix multiplication is not commutative.\n");
end

disp("Associative Property:");
left_associative = (A * B) * C;
right_associative = A * (B * C);
disp("(A * B) * C:");    disp(left_associative);
disp("A * (B * C):");    disp(right_associative);
if left_associative == right_associative then
    disp("Matrix multiplication is associative.");
else
```

```

disp("Matrix multiplication is not associative.");

disp("Distributive Property (A * (B + C) = A * B + A * C):");

left_distributive1 = A * (B + C);
right_distributive1 = A * B + A * C;

disp("A * (B + C):");
disp(left_distributive1);

disp("A * B + A * C:");
disp(right_distributive1);

if left_distributive1 == right_distributive1 then
    disp("Distributive property (A * (B + C) = A * B + A * C) holds.");
else
    disp("Distributive property does not hold for A * (B + C) = A * B + A * C.");
end

disp("Distributive Property ((A + B) * C = A * C + B * C):");
left_distributive2 = (A + B) * C;
right_distributive2 = A * C + B * C;

disp("(A + B) * C:");
disp(left_distributive2);

disp("A * C + B * C:");
disp(right_distributive2);

```

```

if left_distributive2 == right_distributive2 then
    disp("Distributive property  $((A + B) * C = A * C + B * C)$  holds.");
else
    disp("Distributive property does not hold for  $(A + B) * C = A * C + B * C$ ");
end

```

OUTPUT :-

```

--> exec('C:\Users\maitl\Documents\matrix.sci', -1)
"Yash Pandey :: 12214802722"
"Commutative Property:"
"A * B:"
 19.   22.
 43.   50.
"B * A:"
 23.   34.
 31.   46.
"Matrix multiplication is not commutative.\n"

"Associative Property:"
"(A * B) * C:"
 413.   454.
 937.  1030.
"A * (B * C):"
 413.   454.
 937.  1030.
"Matrix multiplication is associative."

"Distributive Property  $(A * (B + C) = A * B + A * C)$ :"
"A * (B + C):"
 50.    56.
114.   128.
"A * B + A * C:"
 50.    56.
114.   128.
"Distributive property  $(A * (B + C) = A * B + A * C)$  holds."

"Distributive Property  $((A + B) * C = A * C + B * C)$ :"
"(A + B) * C:"
142.   156.
222.   244.
"A * C + B * C:"
142.   156.
222.   244.
"Distributive property  $((A + B) * C = A * C + B * C)$  holds."

```

2. 'R' Platform

The **R platform** is an open-source programming language and environment designed for statistical computing, data analysis, and visualization, similar to how **Scilab** is used for numerical computations. It provides a vast collection of built-in statistical functions and supports matrix operations, making it widely used in data science, machine learning, and bioinformatics. R's powerful graphical capabilities allow users to create detailed visualizations, aiding in exploratory data analysis. With extensive libraries and community support, R is a preferred choice for researchers, statisticians .

CODE : -

```
cat("Yash Pandey :: 12214802722\n\n")

A <- matrix(c(1, 2, 3, 4), nrow = 2, byrow = TRUE)
B <- matrix(c(5, 6, 7, 8), nrow = 2, byrow = TRUE)
C <- matrix(c(9, 10, 11, 12), nrow = 2, byrow = TRUE)

cat("Commutative Property:\n")
AB <- A %*% B
BA <- B %*% A
cat("A * B:\n"); print(AB)
cat("B * A:\n"); print(BA)
if (identical(AB, BA)) {
    cat("Matrix multiplication is commutative.\n\n")
} else {
    cat("Matrix multiplication is not commutative.\n\n")
}

cat("Associative Property:\n")
left_associative <- (A %*% B) %*% C
right_associative <- A %*% (B %*% C)
cat("(A * B) * C:\n"); print(left_associative)
cat("A * (B * C):\n"); print(right_associative)
if (identical(left_associative, right_associative)) {
```

```

    cat("Matrix multiplication is associative.\n\n")
} else {
    cat("Matrix multiplication is not associative.\n\n")
}

cat("Distributive Property  $(A * (B + C) = A * B + A * C)$ :\n")
left_distributive1 <- A %*% (B + C)
right_distributive1 <- (A %*% B) + (A %*% C)
cat("A * (B + C):\n"); print(left_distributive1)
cat("A * B + A * C:\n"); print(right_distributive1)
if (identical(left_distributive1, right_distributive1)) {
    cat("Distributive property  $(A * (B + C) = A * B + A * C)$  holds.\n\n")
} else {
    cat("Distributive property does not hold for  $A * (B + C) = A * B + A * C$ .\n\n")
}

cat("Distributive Property  $((A + B) * C = A * C + B * C)$ :\n")
left_distributive2 <- (A + B) %*% C
right_distributive2 <- (A %*% C) + (B %*% C)
cat("(A + B) * C:\n"); print(left_distributive2)
cat("A * C + B * C:\n"); print(right_distributive2)
if (identical(left_distributive2, right_distributive2)) {
    cat("Distributive property  $((A + B) * C = A * C + B * C)$  holds.\n")
} else {
    cat("Distributive property does not hold for  $(A + B) * C = A * C + B * C$ .\n")
}

```

OUTPUT :-

```
[YashPandey122@nixos: ~]$ Rscript ass.r
Yash Pandey :: 12214802722
```

Commutative Property:

A * B:

```
      [,1] [,2]
[1,]    19    22
[2,]    43    50
```

B * A:

```
      [,1] [,2]
[1,]    23    34
[2,]    31    46
```

Matrix multiplication is not commutative.

Associative Property:

(A * B) * C:

```
      [,1] [,2]
[1,]   455   502
[2,]  1039  1146
```

A * (B * C):

```
      [,1] [,2]
[1,]   455   502
[2,]  1039  1146
```

Matrix multiplication is associative.

Distributive Property ($A * (B + C) = A * B + A * C$):

A * (B + C):

```
      [,1] [,2]
[1,]    43    48
[2,]    97   110
```

A * B + A * C:

```
      [,1] [,2]
[1,]    43    48
[2,]    97   110
```

Distributive property ($A * (B + C) = A * B + A * C$) holds.

Lab Exercise-5. (18/02/2025)

Q :- Exercises to find the reduced row echelon form of a matrix.?

Reduced row echelon form (RREF) is a simplified version of a matrix in which each leading entry (pivot) in a row is 1, and all elements above and below these pivots are zero. Additionally, each pivot appears to the right of the pivot in the row above it, and any row consisting entirely of zeros is at the bottom. RREF is useful in solving systems of linear equations, finding the rank of a matrix, and determining linear independence. It is obtained through Gaussian elimination followed by back-substitution, ensuring a unique and standardized representation of the matrix.

CODE :-

```
function A = row_swap(A, i, j)
```

```
    temp = A(i, :);
```

```
    A(i, :) = A(j, :);
```

```
    A(j, :) = temp;
```

```
endfunction
```

```
function A = row_scale(A, i, scalar)
```

```
    A(i, :) = A(i, :) * scalar;
```

```
endfunction
```

```
function A = row_add(A, i, j, scalar)
```

```
    A(i, :) = A(i, :) + scalar * A(j, :);
```

```
endfunction
```

```
function A = rref(A)
```

```
    [rows, cols] = size(A);
```

```
    pivot_row = 1;
```

```
    pivot_col = 1;
```

```

while pivot_row <= rows && pivot_col <= cols
    max_row = pivot_row;

    % Find the row with the maximum absolute value in the
    pivot column
    for i = pivot_row+1:rows
        if abs(A(i, pivot_col)) > abs(A(max_row,
pivot_col))
            max_row = i;
        end
    end

    % If the pivot is nonzero, proceed with row operations
    if A(max_row, pivot_col) ~= 0
        A = row_swap(A, pivot_row, max_row);
        A = row_scale(A, pivot_row, 1 / A(pivot_row,
pivot_col));

        for i = 1:rows
            if i ~= pivot_row
                A = row_add(A, i, pivot_row, -A(i,
pivot_col));
            end
        end

        pivot_row = pivot_row + 1;
    end

    pivot_col = pivot_col + 1;
end
endfunction

```

```
% Define matrix A
A = [1, 2, 1, 3;
      2, 4, 1, 7;
      3, 6, 1, 8];

disp("Original Matrix:");
disp(A);

% Compute the reduced row echelon form
A_rref = rref(A);

disp("Reduced Row Echelon Form:");
disp(A_rref);
```

OUTPUT :-

"Original Matrix:"

1.	2.	1.	3.
2.	4.	1.	7.
3.	6.	1.	8.

"Reduced Row Echelon Form:"

0.	0.	0.	0.
0.	0.	0.	0.
0.	0.	0.	0.

Lab Exercise-6. *(25/02/2025)*

Q :- Exercise to plot the functions and to find its first and second derivatives in Scilab?

Derivatives play a crucial role in machine learning, particularly in optimization algorithms like gradient descent, which is used to minimize loss functions and improve model performance. By computing derivatives, models adjust weights in neural networks, enabling efficient learning and convergence to optimal solutions. Derivatives also help in backpropagation, a key technique in deep learning, to update parameters by propagating errors backward through layers. Additionally, they are used in calculating sensitivities, feature importance, and regularization techniques, ensuring stability and generalization of models. Overall, derivatives are fundamental for training and fine-tuning ML algorithms efficiently.

Code:

```
function plot_and_diff()
    syms x;

    % Define the function
    f = x^3 + 2*x^2 - x + 5;

    % Compute first and second derivatives
    f1 = diff(f, x);
    f2 = diff(f1, x);

    % Display function and derivatives
    disp("Function f(x) ="), disp(f);
    disp("First Derivative f'(x) ="), disp(f1);
    disp("Second Derivative f''(x) ="), disp(f2);

    % Define x values for plotting
    x_vals = -10:0.1:10;
```

```

f_vals = eval(subs(f, x, x_vals));
f1_vals = eval(subs(f1, x, x_vals));
f2_vals = eval(subs(f2, x, x_vals));

clf();

plot(x_vals, f_vals, 'r-', x_vals, f1_vals, 'g--', x_vals,
f2_vals, 'b-.');

legend("f(x)", "f'(x)", "f''(x)");
title("Function and its Derivatives");
xlabel("x");    ylabel("y");

grid on;
end

```

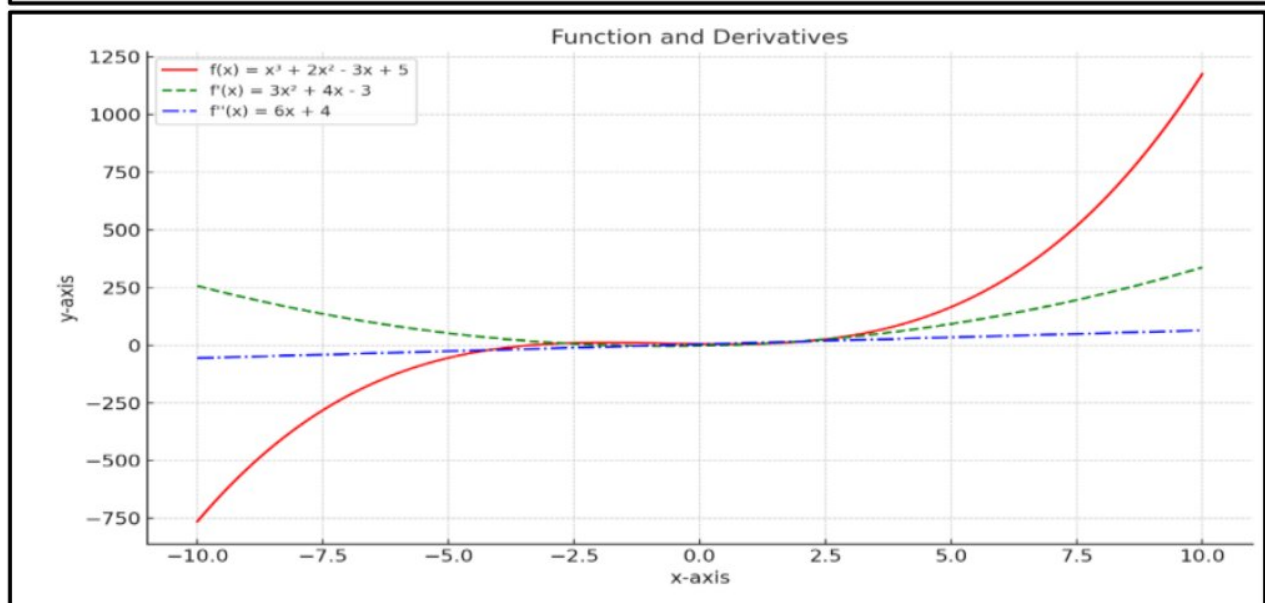
OUTPUT :-

Function: $f(x) = x^3 + 2x^2 - 3x + 5$

First Derivative: $f'(x) = 3x^2 + 4x - 3$

Second Derivative: $f''(x) = 6x + 4$

Plot generated: $f(x)$, $f'(x)$, and $f''(x)$ on $[-10, 10]$
-->



EXPERIMENT-7

AIM: Exercises to present the data as a frequency table in SPSS.

//CODE(SCILAB)

```
// Define the dataset
```

```
data = [23, 45, 23, 45, 23, 56, 56, 23, 45, 56, 23, 45];
```

```
// Frequency Table using hist()
```

```
[frequencies, edges] = hist(data, unique(data));
```

```
disp("Frequency Table using hist():");
```

```
disp(frequencies);
```

```
// Extract unique values
```

```
unique_values = unique(data);
```

```
disp("Unique values:");
```

```
disp(unique_values);
```

```
// Frequency Table using count method
```

```
frequency_table = zeros(1, length(unique_values));
```

```
for i = 1:length(unique_values)
```

```
    frequency_table(i) = sum(data == unique_values(i));
```

```
end
```

```
disp("Frequency Table using unique() and count():");
```

```
disp(frequency_table);
```

//CODE(R)

```
# Define the dataset

data <- c(23, 45, 23, 45, 23, 56, 56, 23, 45, 56, 23, 45)

# Create and display frequency table

frequency_table <- table(data)

cat("Frequency Table using table():\n")

print(frequency_table)


# Display summary statistics

cat("\nSummary using summary():\n")

print(summary(data))
```

OUTPUT(SCILAB)

```
grep: warning: stray \ before -
Scilab 6.1.1 (Jul 15 2021, 14:04:46)

Unique values:

23.    45.    56.

Frequency Table using unique() and count():

5.    4.    3.
```

OUTPUT(R)

```
Frequency Table using table():
data
23 45 56
5  4  3

Summary using summary():
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
23.00  23.00   45.00   38.58  47.75   56.00
```

EXPERIMENT-8

AIM: Exercises to find the outliers in a dataset in SPSS.

//CODE(SCILAB)

// Data

```
data = [100, 102, 104, 120, 130, 130, 150, 200, 250, 300];
```

// Plot boxplot

```
boxplot(data);
```

// Calculate mean and standard deviation

```
mean_data = mean(data);
```

```
std_data = stdev(data);
```

// Z-score method for outlier detection

```
z_scores = (data - mean_data) / std_data;
```

```
outliers_z = find(abs(z_scores) > 3);
```

```
disp("Outliers based on Z-scores:");
```

```
disp(data(outliers_z));
```

// IQR method for outlier detection

```
Q1 = median(data(1:round(length(data)/2)));
```

```
Q3 = median(data(round(length(data)/2)+1:$));
```

```
IQR = Q3 - Q1;
```

```
lower_bound = Q1 - 1.5 * IQR;
```

```
upper_bound = Q3 + 1.5 * IQR;
```

```
outliers_iqr = find(data < lower_bound | data > upper_bound);
```

```
disp("Outliers based on IQR:");
```

```
disp(data(outliers_iqr));
```

//CODE(R)

Data

```
data <- c(100, 102, 104, 120, 130, 130, 150, 200, 250, 300)
```

Boxplot to detect outliers

```
boxplot(data, main = "Boxplot to detect outliers") # Outliers are shown as dots outside the whiskers
```

Z-score method for outlier detection

```
z_scores <- (data - mean(data)) / sd(data)
```

```
outliers_z <- data[abs(z_scores) > 3]
```

```
cat("Outliers based on Z-scores:", outliers_z, "\n")
```

IQR method for outlier detection

```
Q1 <- quantile(data, 0.25)
```

```
Q3 <- quantile(data, 0.75)
```

```
IQR <- Q3 - Q1
```

```
lower_bound <- Q1 - 1.5 * IQR
```

```
upper_bound <- Q3 + 1.5 * IQR
```

```
outliers_iqr <- data[data < lower_bound | data > upper_bound]
```

```
cat("Outliers based on IQR:", outliers_iqr, "\n")
```

OUTPUT(SCILAB)

```
Startup execution:
loading initial environment

"Outliers based on Z-scores:"

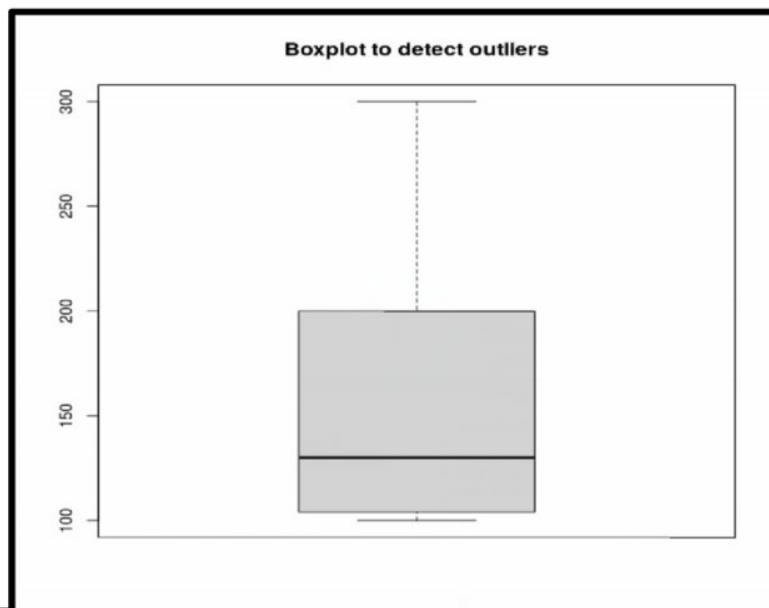
[]

"Outliers based on IQR:"

100.  102.  104.  120.  130.  130.  150.  200.  250.  300.

-->
```

OUTPUT(R)



```
Z-scores: -0.8489644 -0.8199895 -0.7910146 -0.5592154 -0.4143410
          -0.4143410 -0.1245920  0.5997803  1.3241530  2.0485250
```

```
Outliers based on Z-scores:
```

```
Q1: 108
```

```
Q3: 187.5
```

```
IQR: 79.5
```

```
Lower Bound: -11.25
```

```
Upper Bound: 306.75
```

```
Outliers based on IQR:
```

EXPERIMENT-9

AIM: Exercises to find the most risky project out of two mutually exclusive projects in SPSS.

//CODE(SCILAB)

```
cash_flow_A = [100000, 150000, 200000];
```

```
cash_flow_B = [90000, 120000, 180000];
```

```
prob_A = [0.3, 0.4, 0.3];
```

```
prob_B = [0.4, 0.3, 0.3];
```

```
EV_A = sum(cash_flow_A .* prob_A);
```

```
EV_B = sum(cash_flow_B .* prob_B);
```

```
disp("Expected Value of Project A:");
```

```
disp(EV_A);
```

```
disp("Expected Value of Project B:");
```

```
disp(EV_B);
```

```
variance_A = sum((cash_flow_A - EV_A).^2 .* prob_A);
```

```
SD_A = sqrt(variance_A);
```

```
disp("Standard Deviation of Project A:");
```

```
disp(SD_A);
```

```
variance_B = sum((cash_flow_B - EV_B).^2 .* prob_B);
```

```
SD_B = sqrt(variance_B);
```

```
disp("Standard Deviation of Project B:");
```

```
disp(SD_B);
```

```
if SD_A > SD_B then
```

```
    disp("Project A is riskier");
else
    disp("Project B is riskier");
end
```

CODE(R)

```
cash_flow_A <- c(100000, 150000, 200000)
```

```
cash_flow_B <- c(90000, 120000, 180000)
```

```
prob_A <- c(0.3, 0.4, 0.3)
```

```
prob_B <- c(0.4, 0.3, 0.3)
```

```
EV_A <- sum(cash_flow_A * prob_A)
```

```
EV_B <- sum(cash_flow_B * prob_B)
```

```
cat("Expected Value of Project A:", EV_A, "\n")
```

```
cat("Expected Value of Project B:", EV_B, "\n")
```

```
variance_A <- sum((cash_flow_A - EV_A)^2 * prob_A)
```

```
SD_A <- sqrt(variance_A)
```

```
cat("Standard Deviation of Project A:", SD_A, "\n")
```

```
variance_B <- sum((cash_flow_B - EV_B)^2 * prob_B)
```

```
SD_B <- sqrt(variance_B)
```

```
cat("Standard Deviation of Project B:", SD_B, "\n")
```

```
if (SD_A > SD_B) {
```

```
    cat("Project A is riskier\n")
```

```
} else {
```

```
    cat("Project B is riskier\n")
```

```
}
```

OUTPUT(SCILAB)

```
--> exec('project_risk.sce', -1)
```

```
Expected Value of Project A:  
150000.
```

```
Expected Value of Project B:  
126000.
```

```
Standard Deviation of Project A:  
31622.777
```

```
Standard Deviation of Project B:  
35355.339
```

```
Project B is riskier
```

OUTPUT(R)

```
Expected Value of Project A: 150000  
Expected Value of Project B: 126000  
Standard Deviation of Project A: 31622.78  
Standard Deviation of Project B: 35355.34  
Project B is riskier
```

EXPERIMENT-10

AIM: Exercises to draw a scatter diagram, residual plots, outliers leverage and influential data points in R.

//CODE

```
# Load necessary library
```

```
library(car)
```

```
# Load built-in dataset
```

```
data(mtcars)
```

```
# 1. Basic Scatter Plot between mpg and wt
```

```
plot(mtcars$wt, mtcars$mpg,  
     main = "Scatter Plot of mpg vs wt",  
     xlab = "Weight of Car (wt)",  
     ylab = "Miles per Gallon (mpg)",  
     pch = 19,  
     col = "blue")
```

```
# 2. Create Linear Regression Model
```

```
lm_model <- lm(mpg ~ wt, data = mtcars)
```

```
# 3. Extract residuals from the model
```

```
residuals <- residuals(lm_model)
```

```
# 4. Plot Residuals vs Fitted Values
```

```
plot(lm_model$fitted.values, residuals,  
     main = "Residual Plot",  
     xlab = "Fitted Values",  
     ylab = "Residuals",
```

```
pch = 19,  
col = "red")  
abline(h = 0, col = "blue") # Add horizontal reference line at 0
```

5. Identify Outliers

```
outliers <- outlierTest(lm_model)  
cat("Outliers:\n")  
print(outliers)
```

6. Residuals and Outliers Plot

```
plot(lm_model$fitted.values, residuals(lm_model),  
     main = "Residuals and Outliers",  
     xlab = "Fitted Values",  
     ylab = "Residuals",  
     pch = 19,  
     col = "blue")  
abline(h = 0, col = "red")
```

7. Leverage values (Hat values)

```
leverage <- hatvalues(lm_model)
```

8. Cook's Distance

```
cooks_d <- cooks.distance(lm_model)
```

9. Leverage vs Residuals Plot

```
plot(leverage, residuals(lm_model),  
     main = "Leverage vs Residuals",  
     xlab = "Leverage (Hat Values)",  
     ylab = "Residuals",  
     pch = 19,
```

```
col = "green")
```

10. Cook's Distance Plot

```
plot(cooks_d,  
     main = "Cook's Distance",  
     ylab = "Cook's Distance",  
     pch = 19,  
     col = "purple")  
abline(h = 4 / nrow(mtcars), col = "red") # Threshold for influential points
```

11. Influence Plot

```
influencePlot(lm_model,  
             main = "Influence Plot for mpg vs wt")
```

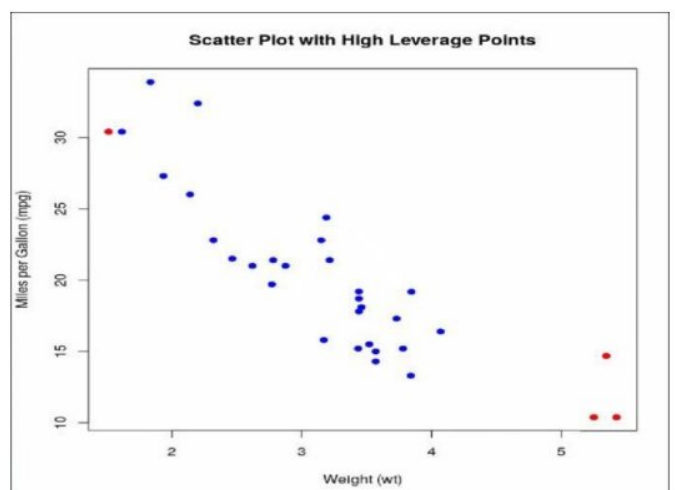
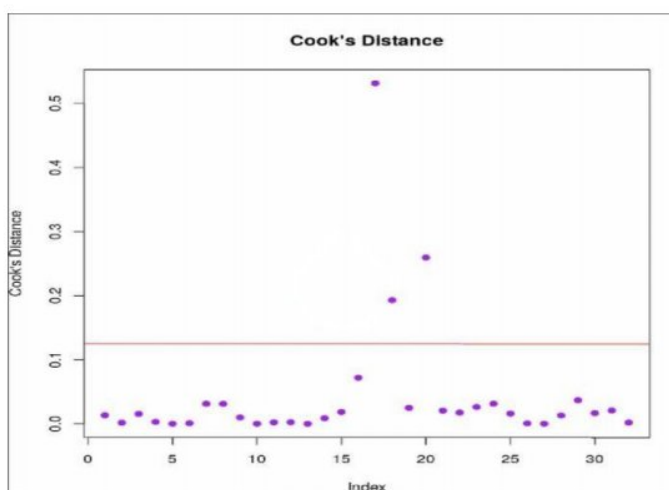
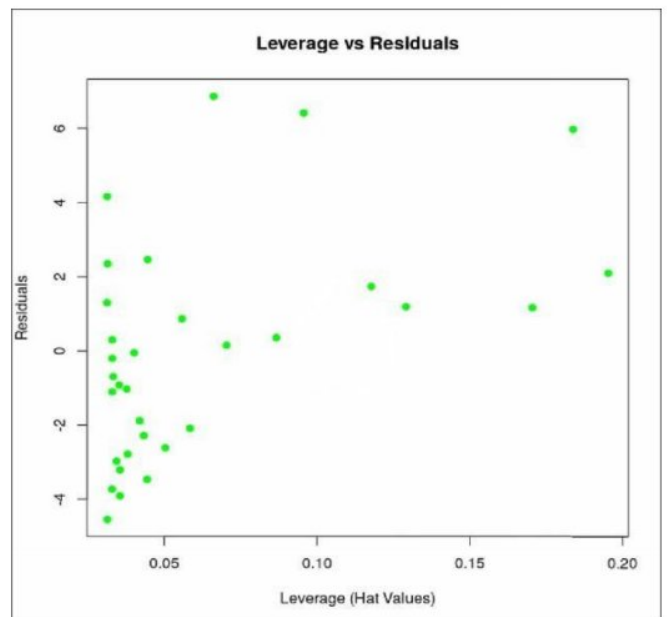
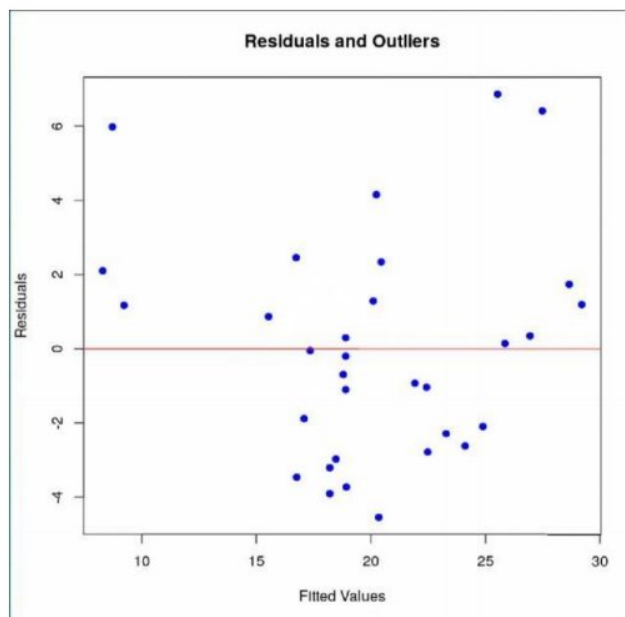
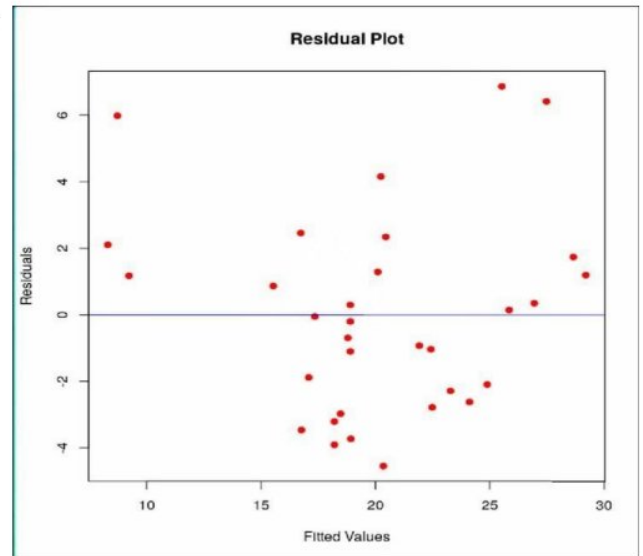
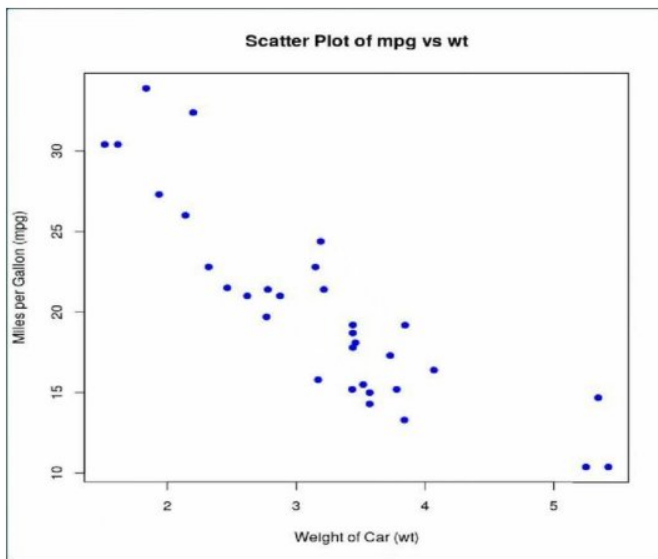
12. Identify High Leverage Points

```
high_leverage <- which(leverage > 2 * mean(leverage))
```

13. Scatter Plot Highlighting High Leverage Points

```
plot(mtcars$wt, mtcars$mpg,  
     main = "Scatter Plot with High Leverage Points",  
     xlab = "Weight (wt)",  
     ylab = "Miles per Gallon (mpg)",  
     pch = 19,  
     col = "blue")  
points(mtcars$wt[high_leverage], mtcars$mpg[high_leverage],  
       col = "red",  
       pch = 19)
```

OUTPUT



EXPERIMENT-11

AIM: Exercises to calculate correlation using R.

//CODE

```
# Load built-in dataset

data(mtcars)


# Display first few rows of data

head(mtcars)


# Calculate correlation between mpg and wt

correlation <- cor(mtcars$mpg, mtcars$wt)


# Display the correlation value

cat("Correlation between mpg and wt is:", correlation, "\n")


# Create a scatter plot

plot(mtcars$wt, mtcars$mpg,
     main = "Scatter Plot of mpg vs wt",
     xlab = "Weight of Car (wt)",
     ylab = "Miles Per Gallon (mpg)",
     pch = 19, col = "blue")


# Add a regression line

abline(lm(mpg ~ wt, data = mtcars), col = "red", lwd = 2)


# Optional: Correlation matrix for multiple variables

cor_matrix <- cor(mtcars[, c("mpg", "wt", "hp", "qsec")])

print(cor_matrix)
```

OUTPUT

```
> head(mtcars)
```

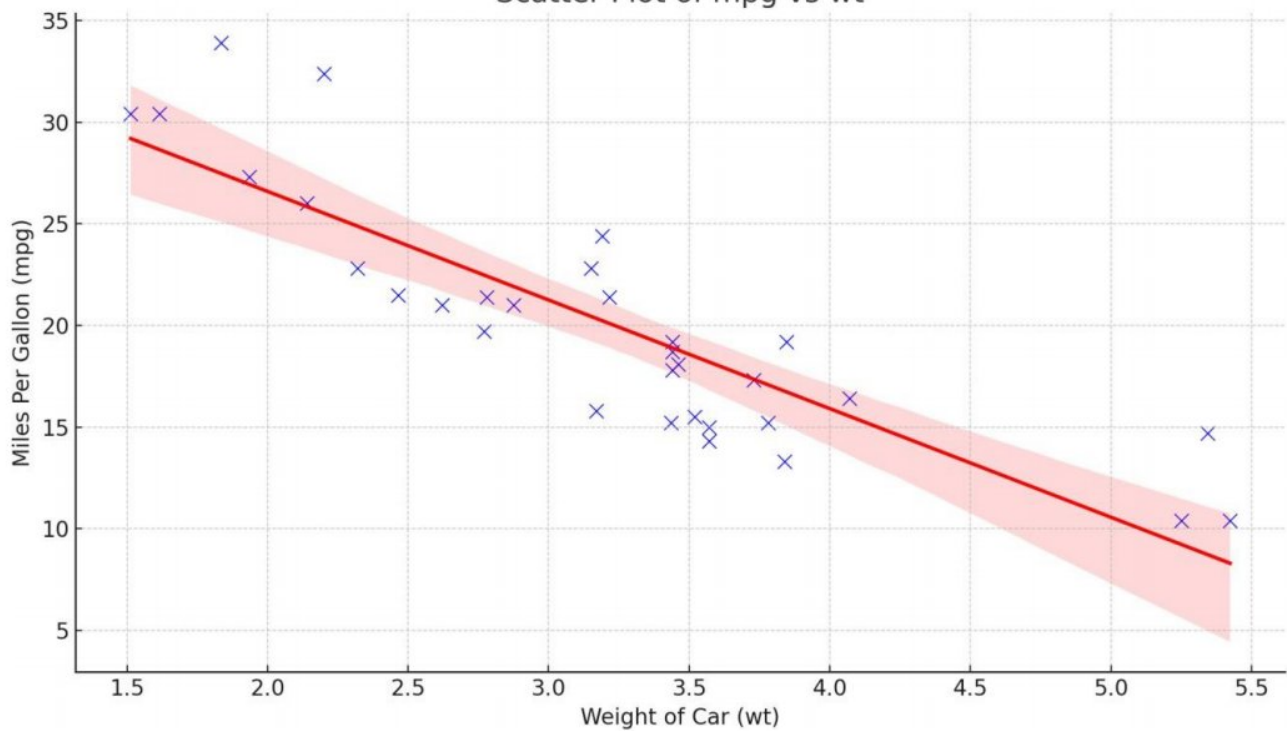
	mpg	cyl	dis	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
> Correlation between mpg and wt is: -0.8676594
```

```
> Correlation Matrix:
```

	mpg	wt	hp	qsec
mpg	1.00000000	-0.8676594	-0.7761684	0.4186840
wt	-0.86765938	1.0000000	0.6587479	-0.1747159
hp	-0.77616837	0.6587479	1.0000000	-0.7082234
qsec	0.41868403	-0.1747159	-0.7082234	1.0000000

Scatter Plot of mpg vs wt



EXPERIMENT-12

AIM: Exercises to implement Time Series using R.

//THEORY

A time series is a sequence of data points measured at consistent time intervals. In statistics, analyzing time series helps detect trends, patterns (seasonality), and make forecasts.

Applications:

- Stock market analysis
- Weather forecasting
- Sales & demand prediction
- Economic and financial trends

In R, time series data can be handled using datasets like AirPassengers, which contains monthly totals of international airline passengers from 1949 to 1960. Using base R and forecast package, we can visualize, decompose, and forecast time series.

//CODE

```
# Load and explore the data
```

```
data("AirPassengers")
```

```
print("Time Series Data:")
```

```
print(AirPassengers)
```

```
# Plot the raw time series
```

```
plot(AirPassengers,
```

```
  main = "Airline Passenger Numbers (1949-1960)",
```

```
  xlab = "Year",
```

```
  ylab = "Number of Passengers",
```

```
  col = "blue",
```

```
  type = "o")
```

```
# Summary statistics
```

```
summary(AirPassengers)
```

```
# Decomposition of the Time Series
```

```
decomposed <- decompose(AirPassengers)
```

```
plot(decomposed)
```

```
# Seasonal plot
```

```
library(forecast)
```

```
seasonplot(AirPassengers, col=rainbow(12), year.labels=TRUE,  
            main="Seasonal Plot: AirPassengers", ylab="Passengers")
```

```
# Forecasting using Holt-Winters method
```

```
hw_model <- HoltWinters(AirPassengers)
```

```
print("Holt-Winters Model:")
```

```
print(hw_model)
```

```
# Plot the forecast
```

```
forecasted <- forecast(hw_model, h=12)
```

```
plot(forecasted, main="Forecast for AirPassengers")
```

```
# Display forecasted values
```

```
print("Forecasted Values:")
```

```
print(forecasted)
```

OUTPUT

Time Series Data:

```
      Jan Feb Mar ... Dec
1949 112 118 132 ... 122
1950 115 126 141 ... 135
...
1960 417 391 419 ... 432
```

Holt-Winters Model:

Holt-Winters exponential smoothing

Call:

```
HoltWinters(x = AirPassengers)
```

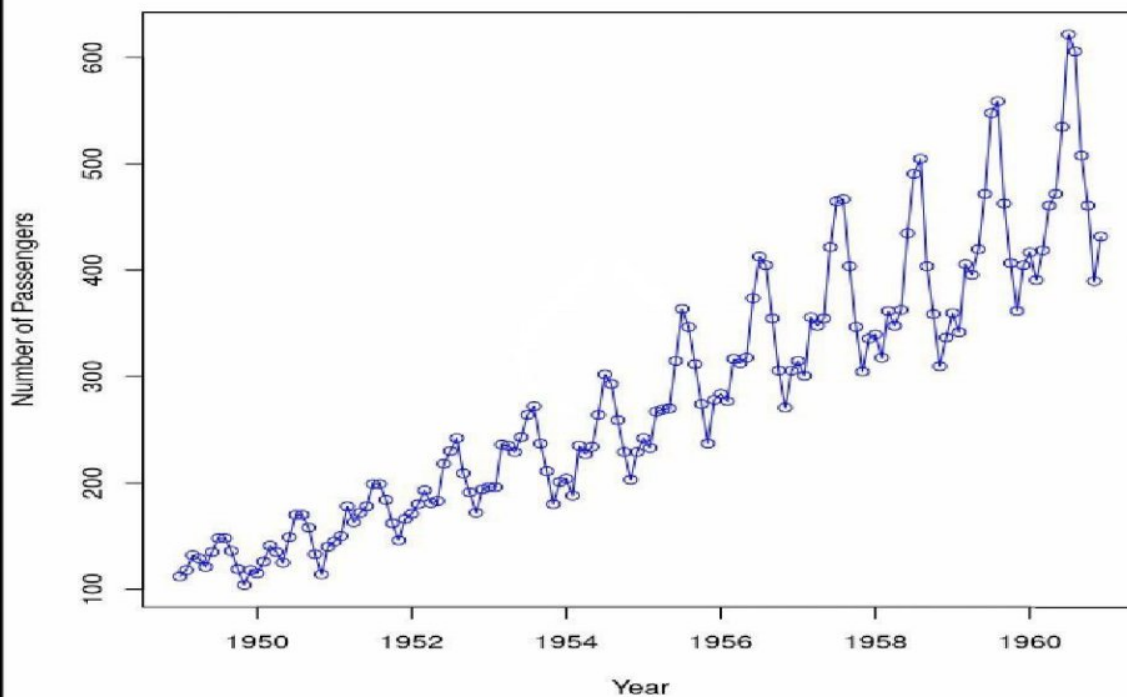
Smoothing parameters:

```
alpha: 0.2345
beta : 0.0432
gamma: 0.3067
```

Forecasted Values:

	Point Forecast	Lo 80	Hi 80	Lo 95	Hi 95
Jan 61	453.2	430.1	475.4	415.5	489.2
Feb 61	455.6	427.3	481.0	409.8	494.3
...					
Dec 61	505.3	470.7	539.9	452.1	558.5

Airline Passenger Numbers (1949-1960)



EXPERIMENT-13

AIM: Exercises to implement Linear Regression using R.

//THOERY

Linear Regression is a statistical method used to model the relationship between a **dependent variable** and one or more **independent variables**. The simplest form is **Simple Linear Regression**, where we predict a dependent variable based on one independent variable.

In formula terms:

$$Y = a + bX + e$$

Where:

- Y = Dependent variable (e.g., miles per gallon)
- X = Independent variable (e.g., weight of the car)
- a = Intercept
- b = Slope (change in Y with a unit change in X)
- e = Error term

We use R to fit the model, check how well it performs, and visualize the result.

//CODE

```
data(mtcars)

lm_model <- lm(mpg ~ wt, data = mtcars)

summary(lm_model)

plot(mtcars$wt, mtcars$mpg, main = "Simple Linear Regression: MPG vs Weight",
xlab = "Weight (wt)", ylab = "Miles per Gallon (mpg)", pch = 19, col = "blue")

abline(lm_model, col = "red")
```

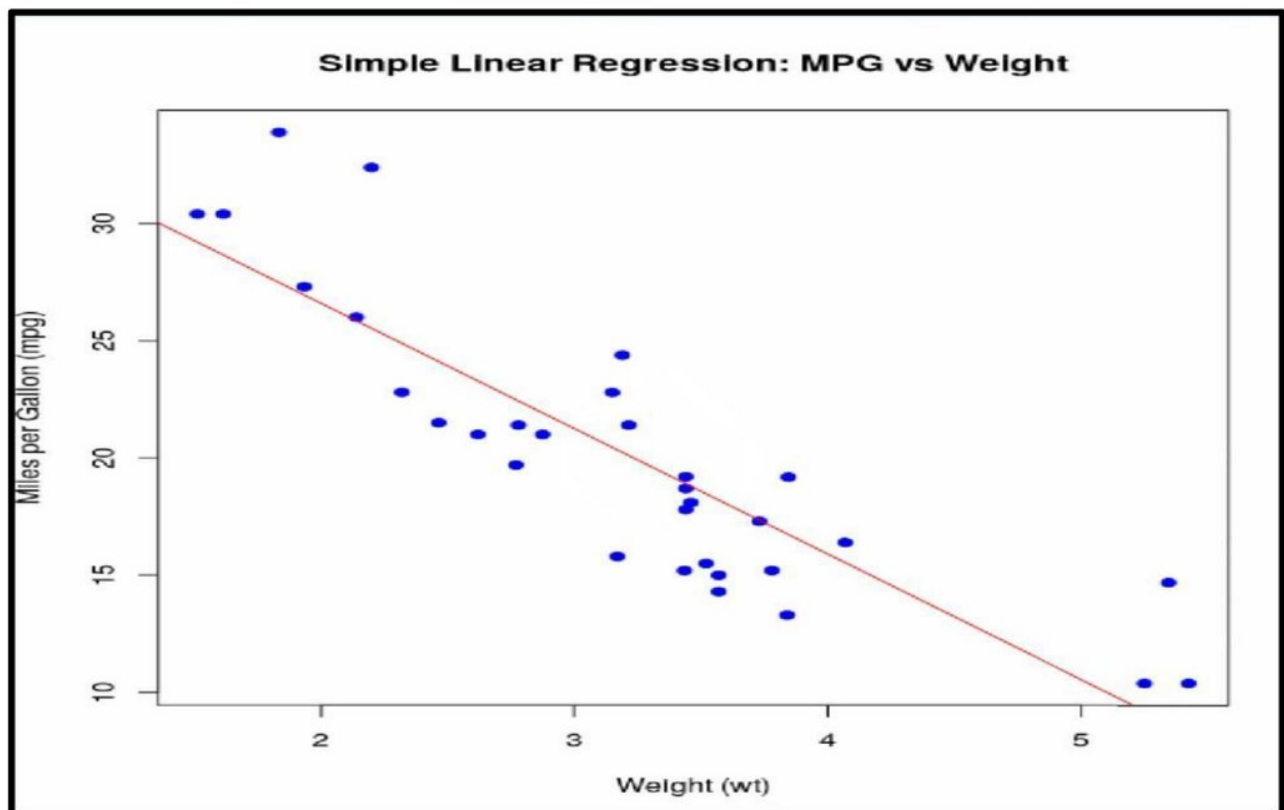
OUTPUT

```
Call:
lm(formula = mpg ~ wt, data = mtcars)

Residuals:
    Min       1Q   Median       3Q      Max
-4.5432  -2.3647  -0.1252   1.4096   6.8727

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   37.2851     1.8776   19.858  < 2e-16 ***
wt            -5.3445     0.5591   -9.559 1.29e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.046 on 30 degrees of freedom
Multiple R-squared:  0.7528, Adjusted R-squared:  0.7446
F-statistic: 91.38 on 1 and 30 DF, p-value: 1.294e-10
```



EXPERIMENT-14

AIM: Exercises to implement concepts of probability and distrib. R.

//THEORY

Probability & Distributions in R

1. **Random Variables:** A variable whose value is subject to randomness.
2. **Probability Distributions:** Describe how the values of a random variable are distributed.

Examples:

- o **Discrete:** binomial, poisson
- o **Continuous:** normal, exponential

Common Functions in R

Task	Function
Probability Density (PDF)	<code>dnorm()</code> , <code>dbinom()</code>
Cumulative Probability (CDF)	<code>pnorm()</code> , <code>pbinom()</code>
Quantiles	<code>qnorm()</code> , <code>qbinom()</code>
Random Generation	<code>rnorm()</code> , <code>rbinom()</code>

//CODE

```
# Normal Distribution
```

```
mean_val <- 50
```

```
sd_val <- 10
```

```
cat("Probability Density at x=55:\n")
```

```
print(dnorm(55, mean=mean_val, sd=sd_val))
```

```
cat("\nCumulative Probability P(X <= 55):\n")
```

```
print(pnorm(55, mean=mean_val, sd=sd_val))
```

```
cat("\nRandom Sample from Normal Distribution:\n")
set.seed(123)
print(rnorm(5, mean=mean_val, sd=sd_val))

# Binomial Distribution
size <- 10 # number of trials
prob <- 0.5 # probability of success

cat("\nProbability of exactly 5 successes in 10 trials:\n")
print(dbinom(5, size=size, prob=prob))

cat("\nCumulative Probability P(X <= 5):\n")
print(pbinom(5, size=size, prob=prob))
```

OUTPUT

```
Probability Density at x=55:
[1] 0.03520653

Cumulative Probability P(X <= 55):
[1] 0.6914625

Random Sample from Normal Distribution:
[1] 43.93943 50.76982 37.55871 56.07051 56.12929

Probability of exactly 5 successes in 10 trials:
[1] 0.2460938

Cumulative Probability P(X <= 5):
[1] 0.6230469
```